S.No: 1	Exp. Name: ***Default Values of Primitive Datatypes using Static Keyword***	**Date: 2026-07-27**

Aim:

Write a Java program that demonstrates the default values of different data types.

When instance variables or static variables are declared without explicit initialization, Java automatically assigns default values.

Your task is to:

- Declare static variables of the following primitive data types (**byte**, **short**, **int**, **long**, **float**, **double**, **char**, **boolean**) and one reference type (**String**) without assigning any values.
- Print the default values of these variables.

Input Format:

- There is no input required for this program.

Output Format:

- The program should display the default values of all primitive data types and the **String** reference in the following format:

```
byte: <value>
short: <value>
int: <value>
long: <value>
float: <value>
double: <value>
char: [<value>]
boolean: <value>
String: <value>
```

Note: Print all static default values by adding their corresponding variable names after each concatenation operator (+).

Source Code:

Q69851/DefaultValues.java

```
package Q69851;
public class DefaultValues {
    static byte defaultByte;
    static short defaultShort;
    static int defaultInt;
        static long defaultLong;
    static float defaultFloat;
    static double defaultDouble;
    static char defaultChar;
    static boolean defaultBoolean;
    static String defaultString;

    public static void main(String[] args) {

        System.out.println("byte: " + defaultByte);
        System.out.println("short: " +   defaultShort );
        System.out.println("int: " + defaultInt     );
        System.out.println("long: " +  defaultLong   );
        System.out.println("float: " + defaultFloat   );
        System.out.println("double: " +defaultDouble   );
        System.out.println("char: ["+defaultChar   +"]");
        System.out.println("boolean: " +defaultBoolean );
        System.out.println("String: " +defaultString   );
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
byte: 0
short: 0
int: 0
long: 0
float: 0.0
double: 0.0
char: []
boolean: false
String: null

S.No: 2	Exp. Name: ***Roots of the quadratic equation***	**Date: 2026-08-03**

Aim:

Write a Java Program to find **Roots** of a Quadratic Equation.

Refer to the displayed sample test cases to strictly match the input and output layout.

Source Code:

q27331/QuadraticEquation.java

```
package q27331;
import java.util.Scanner;
public class QuadraticEquation{
        public static void main(String args[])
        {
        Scanner n=new Scanner(System.in);
        System.out.print("Coefficient a: ");
        double a=n.nextDouble();
        System.out.print("Coefficient b: ");
        double b=n.nextDouble();
        System.out.print("Coefficient c: ");
        double c=n.nextDouble();
        double d=(b*b)-(4*a*c);
        if(d>0)
        {
                System.out.println("The roots are real and distinct");
                double r1=(-b+Math.sqrt(d))/(2*a);
                double r2=(-b-Math.sqrt(d))/(2*a);
                System.out.println("Root1: " +r1+ " Root2: "+r2);
        }
                else if(d==0)
                {
                        System.out.println("The roots are real and
equal");
                        double r=(-b)/(2*a);
                        System.out.println("Root: "+r);
                }
                else{
                        System.out.println("The roots are imaginary");
                }
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Coefficient a:
1

Coefficient b:
6
Coefficient c:
9
The roots are real and equal
Root: -3.0

Test Case - 2
User Output
Coefficient a:
1
Coefficient b:
5
Coefficient c:
8
The roots are imaginary

Test Case - 3
User Output
Coefficient a:
2
Coefficient b:
6
Coefficient c:
1
The roots are real and distinct
Root1: -0.17712434446770464 Root2: -2.8228756555322954

Test Case - 4
User Output
Coefficient a:
2
Coefficient b:
6
Coefficient c:
4

```
Root1: -1.0 Root2: -2.0
```

S.No: 3	Exp. Name: ***Binary Search***	Date: 2026-08-10

Aim:

Write a program to check whether a given element is present or not in a sorted array using the Binary Search technique.

Input Format:

- The first line of input contains an integer N, representing the number of elements in the array.
- The second line of input contains N space-separated integers representing the elements of the array.
- The last line of input contains an integer representing the element to be searched.

Output Format:

- If the element is found, print the index.
- If the element is not found, print "Not found".

Sample Test Case:

Input:

```
7
1 2 3 3 4 5 6
3
```

Output:

```
2
```

Note:

- If duplicate elements are present, print the index of the first occurrence.
- The array is always given in sorted order.
- Refer to the visible test cases for better understanding.

Source Code:

q17128/CTJ17128.java

```
package q17128;
import java.util.Scanner;
public class CTJ17128{
	public static void main(String args[])
	{
		Scanner s=new Scanner(System.in);
		int n=s.nextInt();
		int []a=new int[n];
		for(int i=0;i<n;i++)
			{
				a[i]=s.nextInt();
			}
		int key=s.nextInt();
		int l=0;
		int h=n-1;
		int result=-1;
		while(l<=h)
			{
				int mid=(l+h)/2;
				if(a[mid]==key)
				{
					result=mid;
					h=mid-1;
				}
				else if(a[mid]<key)
				{
					l=mid+1;
				}
				else{
					h=mid-1;
				}
			}
				if(result!=-1)
				{
					System.out.println(result);
				}
		else{
			System.out.println("Not found");
		}
		s.close();
	}

}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
7
1 2 3 3 4 5 6
3
2

Test Case - 2
User Output
10
1 2 3 4 5 6 7 8 9 19
20
Not found

S.No: 4	Exp. Name: ***Bubble Sort Technique***	Date: 2026-08-10

Aim:

Write a Java program to sort an array of integers using the bubble sort algorithm. The program should read the number of elements and the array elements from the user, then display the sorted array in ascending order.

Input Format:

- The first line contains an integer n representing the number of elements in the array.
- The second line contains n space-separated integers representing the array elements.

Output Format:

- Print the sorted array elements in ascending order, separated by a space:

```
<int1> <int2> <int3> ...
```

Note:

- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q32083/BubbleSort.java

```
package q32083;
import java.util.Scanner;
public class BubbleSort {
        //static int temp;
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        bubbleSort(arr);

        for (int num : arr) {
            System.out.print(num + " ");
        }

        scanner.close();
    }

    public static void bubbleSort(int[] arr) {
        for(int i=0;i<arr.length-1;i++)
                        {
                                for(int j=0;j<arr.length-i-1;j++)
                                        {
                                        if(arr[j]>arr[j+1]){
                                                int temp=arr[j];
                                        arr[j]=arr[j+1];
                                        arr[j+1]=temp;
                                        }
                        }
        }



        //Type your content here
```

```
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
12 35 69 48 51
12 35 48 51 69

Test Case - 2
User Output
3
5 -1 25
-1 5 25

S.No: 5	Exp. Name: ***Removing duplicate characters from a glitchy message by using StringBuffer constructor***	**Date: 2026-08-10**

Aim:

A group of friends sent a message to Jaya, but due to a technical glitch, the message contained duplicate characters. To overcome this kind of problem she decided to craft a Java program using a **StringBuffer** constructor that takes the glitchy message as input and returns a new string with all duplicate characters removed.

How can Jaya craft this program?

Input Format:

The input line reads a string representing the glitchy message.

Output Format:

The output is a string after removing the duplicate characters (Including special characters, space).

Note: Use println() to print the output statements.

Source Code:

q22135/Main.java

```
package q22135;
import java.util.Scanner;
public class Main{
        public static void main(String args[])
                {
                Scanner sc=new Scanner(System.in);
String input=sc.nextLine();
StringBuffer sb=new StringBuffer(input);
for(int i=0;i<sb.length();i++)
        {
                for(int j=i+1;j<sb.length();j++){
                if(sb.charAt(i)==sb.charAt(j))
                {
                        sb.deleteCharAt(j);
                        j--;
                }
                }
        }
System.out.println(sb.toString());
}
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
abcdef
abcdef

Test Case - 2
User Output
a a b b c c
a bc

S.No: 6	Exp. Name: ***Constructor Overloading***	**Date: 2026-08-17**

Aim:

Write a Java program that demonstrates constructor overloading in a class named **Book**. The **Book** class should have the following attributes:

- **title** - String
- **author** - String
- **pageCount** - int

Implement the following constructors in the **Book** class:

1. A default constructor that initializes the title and author to "Unknown Title" and "Unknown Author," respectively, and sets **pageCount** to 0.
2. A constructor that takes **title** and **author** as parameters and initializes the corresponding attributes. The **pageCount** should be set to 0.
3. A constructor that takes **title, author**, and **pageCount** as parameters and initializes the corresponding attributes.

Additionally, include a method named **displayBookDetails** that prints the details of the book, including the **title, author,** and **pageCount.**

Sample Input and Output format:

```
Details for Book 1:
Title: Introduction to Java Programming
Author: John Smith
Details for Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson
Page Count: 350
Book 1:
Title: Introduction to Java Programming
Author: John Smith
Page Count: 0
Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson
Page Count: 350
```

Source Code:

Book.java

```
import java.util.Scanner;

public class Book {
String title;
        String author;
        int pageCount;
        Book()
        {
                title="Unknown Title";
                author="Unknown Author";
                pageCount=0;
        }
        Book(String title,String author)
        {
                this.title=title;
                this.author=author;
                this.pageCount=0;
        }
        Book(String title,String author,int pageCount)
        {
        this.title=title;
this.author=author;
this.pageCount=pageCount;
}
void displayBookDetails(){
        System.out.println("Title: "+title);
        System.out.println("Author: "+author);
System.out.println("Page Count: "+pageCount);
        }

//write your code here..



    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking user inputs for Book 1
        System.out.println("Details for Book 1:");
        System.out.print("Title: ");
        String title1 = scanner.nextLine();
        System.out.print("Author: ");
        String author1 = scanner.nextLine();
```

```
        // Creating Book 1 using the constructor with title and author
parameters
        Book book1 = new Book(title1, author1);

        // Taking user inputs for Book 2
        System.out.println("Details for Book 2:");
        System.out.print("Title: ");
        String title2 = scanner.nextLine();
        System.out.print("Author: ");
        String author2 = scanner.nextLine();
        System.out.print("Page Count: ");
        int pageCount2 = scanner.nextInt();

        // Creating Book 2 using the constructor with all parameters
        Book book2 = new Book(title2, author2, pageCount2);

        // Displaying details of each book
        System.out.println("Book 1:");
        book1.displayBookDetails();

        System.out.println("Book 2:");
        book2.displayBookDetails();

        // Close the scanner
        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Details for Book 1:
Title:
Introduction to Java Programming
Author:
John Smith
Details for Book 2:
Title:
Data Structures and Algorithms
Author:

Alice Johnson
Page Count:
350
Book 1:
Title: Introduction to Java Programming
Author: John Smith
Page Count: 0
Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson
Page Count: 350

Test Case - 2
User Output
Details for Book 1:
Title:
Programming Basics
Author:
Jane Doe
Details for Book 2:
Title:
Advanced Java Concepts
Author:
Bob Williams
Page Count:
500
Book 1:
Title: Programming Basics
Author: Jane Doe
Page Count: 0
Book 2:
Title: Advanced Java Concepts
Author: Bob Williams
Page Count: 500

S.No: 7	Exp. Name: ***Implementing Method overloading***	**Date: 2026-08-24**

Aim:

Write a Java program with a class name OverloadArea with overload methods area(float) and area(float, float) to find area of square and rectangle.

Write the main method within the class and assume that it will receive a total of 2 command line arguments of type float.

If the main() is provided with arguments : 1.34, 1.98 then the program should print the output as:
Area of square for side in meters 1.34 : 1.7956
Area of rectangle for length and breadth in meters 1.34, 1.98 : 2.6532001

Source Code:

OverloadArea.java

```
class OverloadArea
	{
		float area(float side)
		{
			return side*side;
		}
		float area(float length,float breadth)
		{
			return length*breadth;
		}
		public static void main(String args[])
		{
			float side=Float.parseFloat(args[0]);
			float length=Float.parseFloat(args[0]);
			float breadth=Float.parseFloat(args[1]);
			OverloadArea obj=new OverloadArea();
			float squareArea=obj.area(side);
			float rectangleArea=obj.area(length,breadth);
			System.out.printf("Area of square for side in
meters %.2f: %.4f",side,squareArea);
			System.out.printf("\n");
			System.out.printf("Area of rectangle for length
and breadth in meters %.2f, %.2f: %.7f",length,breadth,rectangleArea);
			System.out.printf("\n");
		}
	}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Area of square for side in meters 1.34: 1.7956
Area of rectangle for length and breadth in meters 1.34, 1.98: 2.6532001

Test Case - 2
User Output
Area of square for side in meters 2.30: 5.2900
Area of rectangle for length and breadth in meters 2.30, 2.80: 6.4399996

S.No: 8	Exp. Name: ***Basics of Class and Object***	**Date: 2026-08-17**

Aim:

The Spy Academy is implementing a new system to manage information about their agents. As part of this system, they need to create a class to represent an agent. Imagine yourself as the chief developer, and your task is to design a basic class named **Agent**.

Task:

1. Design a class named **Agent** with the following attributes:
   - **name** (String)
   - **age** (int)
   - **codename** (String)
   - **isUndercover** (boolean)
2. Implement a method named **introduce()** that prints a message introducing the agent, including their **name**, **age**, and **codename**.

Additional Instructions:

- In the main method, create an instance of the **Agent** class named **topAgent**.
- Use user input within the main method to set the attributes of **topAgent** (**name**, **age**, **codename**, and **isUndercover**).
- Invoke the **introduce()** method on **topAgent** to introduce this top agent to the team.

Note:

- Partial code is given fill in the remaining code.

Source Code:

q22852/SpySystem.java

```
package q22852;
import java.util.Scanner;

// create class Agent and Attributes
class Agent{
        String name;
        int age;
        String codename;
        boolean isUndercover;


    // create Method to introduce the agent

    void introduce() {
        System.out.println("Hello, I am Agent " +codename+ "."); //
write your code here
        System.out.println("My real name is " +name+ "."); // write
your code here
        System.out.println("I am " + age+ " years old."); // write your
code here
    }
}

public class SpySystem {
    public static void main(String[] args) {
Agent topAgent=new Agent();

        Scanner scanner = new Scanner(System.in);
        System.out.print("name: ");
        topAgent.name = scanner.nextLine();
        System.out.print("age: ");
        topAgent.age = scanner.nextInt();
        scanner.nextLine();
        System.out.print("codename: ");
        topAgent.codename = scanner.nextLine();
        System.out.print("undercover? (true/false): ");
        topAgent.isUndercover = scanner.nextBoolean();


        topAgent.introduce();

        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
name:
Alice
age:
27
codename:
007
undercover? (true/false):
true
Hello, I am Agent 007.
My real name is Alice.
I am 27 years old.

Test Case - 2
User Output
name:
Bob
age:
25
codename:
117
undercover? (true/false):
false
Hello, I am Agent 117.
My real name is Bob.
I am 25 years old.

S.No: 9	Exp. Name: ***Constructor Overloading and Default Constructor***	**Date: 2026-08-24**

Aim:

You are given a Java program that demonstrates the concept of constructor overloading.

The class **Test** contains:

1. Two private integer variables i and j.
2. Three overloaded constructors:
   - A **default constructor** that sets both i and j to 0.
   - A **constructor with one parameter** that sets i to the given value and j to 0.
   - A **constructor with two parameters** that sets both i and j using the given values.
   - Implement a method **display()** that prints the values of i and j.
3. The program creates three objects of the **Test** class
   - **obj1** - created using the default constructor.
   - **obj2** - created using the constructor with one input value for i.
   - **obj3** - created using the constructor with two input values for i and j.

Input Format:

- First, **obj1** uses the default constructor - no input.
- After displaying **obj1**, read an integer from the user - this is used for i in **obj2**.
- After displaying **obj2**, read two integers - these are used for i and j in **obj3**.

Output Format:

- The program displays the values of i and j for each object in the following format:

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:<i_value>
Displaying values for obj2:
Value of i: <value>
Value of j: 0
Enter a value for i:<i_value>
Enter a value for j:<j_value>
Displaying values for obj3:
Value of i: <i_value>
Value of j: <j_value>
```

Sample Test Case (Combined Input and Output):

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i: 10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i: 40
Enter a value for j: 50
Displaying values for obj3:
Value of i: 40
Value of j: 50
```

Note:

- Input and output are combined in the test cases. Please ensure your program takes input and displays output simultaneously, in the same sequence as shown in the sample interaction.
- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q21947/Main.java

```
package q21947;
import java.util.Scanner;
class Test {
    // Private instance variables
    private int i;
    private int j;

    // Default constructor
        Test()
        {
                i=0;j=0;
        }
    Test(int i)
        {
                this.i=i;
                this.j=0;
        }
        Test(int i,int j)
        {
                this.i=i;
                this.j=j;
        }
        void display()
        {
                System.out.println("Value of i: "+i);
                System.out.println("Value of j: "+j);
        }




    // Parameterized constructor with one parameter



    // Parameterized constructor with two parameters



    // Display function to print the values of i and j


}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Test obj1 = new Test();
        System.out.println("Displaying values for obj1:");
        obj1.display();
        System.out.print("Enter a value for i: ");
        int value1 = scanner.nextInt();
        Test obj2 = new Test(value1);
        System.out.println("Displaying values for obj2:");
        obj2.display();
        System.out.print("Enter a value for i: ");
        int value2 = scanner.nextInt();
        System.out.print("Enter a value for j: ");
        int value3 = scanner.nextInt();
        Test obj3 = new Test(value2, value3);
        System.out.println("Displaying values for obj3:");
        obj3.display();
        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i:
40
Enter a value for j:
50
Displaying values for obj3:
Value of i: 40

Test Case - 2
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i:
20
Enter a value for j:
30
Displaying values for obj3:
Value of i: 20
Value of j: 30

S.No: 10	Exp. Name: ***Number and Square - Single Inheritance***	**Date: 2026-08-31**

Aim:

Write a Java program to demonstrate **single inheritance** by reading an integer and displaying its square.

Create a class named **NumberIn** with the following members:

- An integer data member **num**.
- A method **inputNum()** that reads an integer from the user and stores it in **num**. The method should display the prompt:

```
Enter number:
```

Create another class named **SquareOut** that extends **NumberIn** and implements the following member:

- A method **displaySquare()** that displays the square of the entered number.

Note: The **MainFunction** class has already been provided in the editor. It creates an object of the **SquareOut** class, invokes the **inputNum()** method to read the input, and then invokes the **displaySquare()** method to display the result. Do not modify the **MainFunction** class. Implement only the **NumberIn** and **SquareOut** classes.

Source Code:

q23112/MainFunction.java

```
package q23112;
import java.util.Scanner;
class SquareOut
        {static int n,r;
                public void inputNum()
                {
                        System.out.print("Enter number: ");
                }
                public void displaySquare(){
                        Scanner s=new Scanner(System.in);
                        int n=s.nextInt();
                        r=n*n;
                        System.out.println(r);
                }
        }
class Numberln extends SquareOut
        {

        }
public class MainFunction {
    public static void main(String[] args) {
        SquareOut squareout = new SquareOut();
        squareout.inputNum();
        squareout.displaySquare();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number:
9
81

Test Case - 2
User Output
Enter number:

15
225

S.No: 11	Exp. Name: ***Multilevel Inheritance in Java***	**Date: 2026-08-31**

Aim:

Write a Java program to show multilevel inheritance to manage student information and calculate their marks. The program should include the following functionalities:

Student Class:

This class should contain two attributes:

- **id (int)**: Represents the student's ID.
- **name (String):** Represents the student's name.

Methods:

- **setData(int id, String name):** To set the student's ID and name.
- **displayData():** To display the student's ID and name.

Marks Class (inherits from Student):

This class should contain the following marks:

- **javaMarks (float):** Marks obtained in Java.
- **cMarks (float):** Marks obtained in C.
- **cppMarks (float):** Marks obtained in C++.

Methods:

- **setMarks(float javaMarks, float cMarks, float cppMarks) :** To set the marks for Java, C, and C++.
- **displayMarks():** To display the marks for Java, C, and C++.

Result Class (inherits from Marks):

This class should calculate the total and average of the marks.

Methods:

- **compute() :** To calculate the total and average marks.
- **showResult()**: To display the total and average marks.

Input Format:

The program should prompt the user for the following inputs:

- Student ID (integer)
- Student Name (string)
- Java Marks (float)
- C Marks (float)
- C++ Marks (float)

Output Format:

The program should output the following:

- Student ID in the format "Id : <id>"
- Student Name in the format "Name : <name>"
- Java Marks in the format "Java marks : <java marks>"
- C Marks in the format "C marks : <c marks>"
- C++ Marks in the format "Cpp marks : <cpp marks>"
- Total Marks in the format "Total : <total>"

• Average Marks in the format "Avg : <avg>"

Source Code:

MultilevelInheritance.java

```
import java.util.Scanner;
class Student{
        int id;
        String name;
        //student(int id,String name)
        //{
        //      this.id=id;
        //      this.name=name;
        //}
        public void setData(int id,String name){
                this.id=id;
                this.name=name;
        }
public void displayData(){
        System.out.println("Id : "+ id);
        System.out.println("Name : "+ name);
}
}
        class Marks extends Student
                {
                 float javaMarks,cMarks,cppMarks;
                        public void setMarks(float javaMarks,float
cMarks,float cppMarks)
                        {
                                this.javaMarks=javaMarks;
                                this.cMarks=cMarks;
                                this.cppMarks=cppMarks;
                        }
                        public void displayMarks()
                        {
                                System.out.println("Java marks :
"+javaMarks);
                                System.out.println("C marks :
"+cMarks);
                                System.out.println("Cpp marks :
"+cppMarks);
                        }
                }
class Result extends Marks{
        float total,average;
        public void compute()
        {
                total=javaMarks+cMarks+cppMarks;
```

```
        public void showResult()
        {
                System.out.println("Total : " + total);
                System.out.println("Avg : " + average);
        }
}
public class MultilevelInheritance{
        public static void main(String args[]) {
            Scanner s=new Scanner(System.in);
            Scanner sc=new Scanner(System.in);
                Result r = new Result();
                System.out.print("Id: ");
                int id=s.nextInt();
                System.out.print("Name: ");
                String name = sc.nextLine();
                r.setData(id, name);
                System.out.print("Java marks: ");
                float jv=s.nextFloat();
                System.out.print("C marks: ");
                float c=s.nextFloat();
                System.out.print("CPP marks: ");
                float cpp=s.nextFloat();
        r.displayData();
                r.setMarks(jv,c,cpp);
                r.displayMarks();
                r.compute();
                r.showResult();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Id:
1001
Name:
Yamuna
Java marks:
75.5

C marks:
96.5
CPP marks:
87
Id : 1001
Name : Yamuna
Java marks : 75.5
C marks : 96.5
Cpp marks : 87.0
Total : 259.0
Avg : 86.333336

Test Case - 2
User Output
Id:
1000
Name:
Ganga
Java marks:
77
C marks:
85.5
CPP marks:
96.5
Id : 1000
Name : Ganga
Java marks : 77.0
C marks : 85.5
Cpp marks : 96.5
Total : 259.0
Avg : 86.333336

S.No: 12	Exp. Name: ***Areas of Shapes Using Abstract Class***	**Date: 2026-09-28**

Aim:

Design an abstract class named **Shape** to represent geometric shapes. The **Shape** class should declare the following abstract methods:

- **double calculateArea()**: Calculates and returns the area of the shape.
- **double calculatePerimeter()**: Calculates and returns the perimeter of the shape.

In addition, implement a concrete method named **displayDetails()** that displays the area and perimeter of the shape. Create a subclass named **Circle** that extends the **Shape** class.

The **Circle** class should:

- Store the radius of the circle.
- Implement the **calculateArea()** method to calculate and return the area of the circle.
- Implement the **calculatePerimeter()** method to calculate and return the perimeter (circumference) of the circle.

Input Format:

- The first line reads an integer representing the radius of the circle on the same line after the prompt:

```
Enter the radius of the circle: <radius>
```

Output Format:

- The output displays the details of the circle in the following format:

```
Shape details:
Area: <area>
Perimeter: <perimeter>
```

Note:

- Use `Math.PI` as the value of π while calculating the area and perimeter of the circle.
- A partial code is provided in the editor. Complete the given code by implementing the required logic.

Source Code:

Circle.java

```
import java.util.Scanner;
abstract class Shape {
        //System.out.println("Shape details: ");
        abstract double calculateArea();
        abstract double calculatePerimeter();
}
class Circle extends Shape {
    double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    @Override
 public double calculateArea() {
         a=Math.PI*radius*radius;
        return a;

    }
    @Override
 public double calculatePerimeter() {
b=2*Math.PI*radius;
         return b;
 }
        double a,b;
        public void displayDetails()
        {
                System.out.println("Shape details:");
                System.out.println("Area: " +calculateArea());
                System.out.println("Perimeter: "+calculatePerimeter());

    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the radius of the circle: ");
        double radius = scanner.nextDouble();
        Circle circle = new Circle(radius);
        circle.displayDetails();
        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the radius of the circle:
3
Shape details:
Area: 28.274333882308138
Perimeter: 18.84955592153876

Test Case - 2
User Output
Enter the radius of the circle:
4.2
Shape details:
Area: 55.41769440932395
Perimeter: 26.389378290154262

S.No: 13	Exp. Name: ***Demonstrating super keyword***	**Date: 2026-09-28**

Aim:

Create a Java Class Using Single Inheritance with a base class Addition consists of method add()and a derived class Calculation consists of method sub, Demonstrate Super keyword by calling add() method first from the base class and then its child class method sub().

Source Code:

q18039/Main.java

```
package q18039;
import java.util.Scanner;
class Addition {

    // write your code here
    void addition(int a,int b){
                System.out.println("Addition: " + (a+b));
        }

}

class Calculation extends Addition {
        void subtraction(int a,int b)
        {
                System.out.println("Subtraction: " + (a-b));
        }void cal(int a,int b)
        {
                super.addition(a,b);
                subtraction(a,b);
        }

    // write your code here


}
class Main{
        public static void main(String args[])
        {
                Scanner sc=new Scanner(System.in);
                int a=sc.nextInt();
                int b=sc.nextInt();
                Calculation c=new Calculation();
                c.cal(a,b);
        }
}
// write your code here
```

Execution Results - All test cases have succeeded!

Test Case - 1

5
3
Addition: 8
Subtraction: 2

Test Case - 2
User Output
10
15
Addition: 25
Subtraction: -5

S.No: 14	Exp. Name: ***Implementation of Multiple Inheritance using Interface***	**Date: 2026-09-28**

Aim:

You are required to implement the **Circle** class, which demonstrates multiple inheritance using interfaces. The provided **Shape** and **Color** interfaces define methods that need to be implemented within the **Circle** class.

1. **Shape** Interface:

The **Shape** Interface contains two methods:

- **double calculateArea():** To calculate the area of the circle.
- **double calculatePerimeter():** To calculate the perimeter of the circle.

The **Color** Interface contains one method:

- **String getColor():** To get the color of the circle.

Circle **Class**

- Create a class named **Circle** that implements both the **Shape** and **Color** interfaces.
- The class should contain:
  - A double variable to store the radius of the circleA String variable to store the color of the circle
- The class must provide method bodies for all methods declared in the interfaces.
- Implement the calculations using the following formulas:
- $Area = \pi \times radius \times radius Perimeter = 2 \times \pi \times radius$

Input Format:

The program prompts the user for input in the following order:

```
Radius of the circle:   //reads a double value
Color of the circle:   //reads a String value
```

Output Format:

- After reading the input, the program prints:

```
Circle Area: <area formatted to two decimal places>
Circle Perimeter: <perimeter formatted to two decimal places>
Circle Color: <color>
```

Note:

- Use **Math.PI** for π value.

Source Code:

q27463/MultipleInheritance.java

```
package q27463;
import java.util.Scanner;

interface Shape {
    double calculateArea();
    double calculatePerimeter();
}
interface Color {
    String getColor();
}
class Circle implements Shape, Color {
    double radius;
        String color;
        Circle(double radius,String color){
                this.radius=radius;
                this.color=color;
        }
        public double calculateArea()
        {
                return Math.PI*radius*radius;
        }
        public double calculatePerimeter()
        {
                return 2*Math.PI*radius;
        }
        public String getColor(){
                return color;
        }
}
public class MultipleInheritance {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Radius of the circle: ");
        double radius = scanner.nextDouble();
        scanner.nextLine();
        System.out.print("Color of the circle: ");
        String color = scanner.nextLine();
        Circle circle = new Circle(radius, color);

        // Formatting double values to 2 decimal places
        System.out.printf("Circle Area: %.2f%n",
circle.calculateArea());
        System.out.printf("Circle Perimeter: %.2f%n",
circle.calculatePerimeter());
```

```
        System.out.println("Circle Color: " + circle.getColor());

        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Radius of the circle:
2.0
Color of the circle:
red
Circle Area: 12.57
Circle Perimeter: 12.57
Circle Color: red

Test Case - 2
User Output
Radius of the circle:
5.0
Color of the circle:
violet
Circle Area: 78.54
Circle Perimeter: 31.42
Circle Color: violet

Test Case - 3
User Output
Radius of the circle:
3.0
Color of the circle:
blue
Circle Area: 28.27

```
Circle Color: blue
```

S.No: 15	Exp. Name: ***Polymorphism - Area of Geometric shapes***	**Date: 2026-09-28**

Aim:

Write a Java program that provides an interactive menu for users to perform calculations on 2D shapes. The program should allow users to calculate the area of different shapes: circle and square. Implement the program using the concept of inheritance and method overriding to achieve runtime polymorphism.

Source Code:

ShapeCalculator.java

```
import java.util.Scanner;

abstract class Shape {
    abstract double calculateArea();
}
class Circle extends Shape{
        double r;
        Circle(double r)
        {
                this.r=r;
        }
        @Override
        double calculateArea(){
                return Math.PI*r*r;
        }
}
class Square extends Shape{
        double s;
        Square(double s){
                this.s=s;
        }
        @Override
        double calculateArea()
        {
                return s*s;
        }
}
class ShapeCalculator
        {
public static void main(String args[]){
                while(true){
                        Scanner sc=new Scanner(System.in);
                        System.out.println("Select an option:\n1. Area
of Circle\n2. Area of Square\n3. Exit");
                        System.out.print("Enter option: ");
                        int n=sc.nextInt();
                        Shape shape;
                        if(n==1){
                        System.out.print("radius: ");
                        double r=sc.nextDouble();
                                shape=new Circle(r);
                                System.out.println("Area: " +
shape.calculateArea());
                        }
```

```
                                else if(n==2){
                                System.out.print("side: ");
                                int s=sc.nextInt();
                                shape=new Square(s);
                                System.out.println("Area:
"+shape.calculateArea());
                                }
                        else if(n==3){
                                return;
                        }
                        else{
                                System.out.println("Invalid choice");
                        }
        }
}
        }
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
1
radius:
3.75
Area: 44.178646691106465
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
2
side:
40
Area: 1600.0
Select an option:

1. Area of Circle
2. Area of Square
3. Exit
Enter option:
5
Invalid choice
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
3

Test Case - 2
User Output
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
1
radius:
3.3578
Area: 35.4208943214851
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
5
Invalid choice
Select an option:
1. Area of Circle
2. Area of Square
3. Exit
Enter option:
3

S.No: 16	Exp. Name: ***Exception Handling***	Date: 2026-10-05

Aim:

Write a Java program that demonstrates the use of Exception Handling for the following scenarios:

1. Divide by zero (`ArithmeticException`)
2. Accessing an invalid index in an array (`ArrayIndexOutOfBoundsException`)
3. Using a method on a null object (`NullPointerException`)

The program should prompt the user for inputs in sequence, perform each operation, and handle exceptions using try-catch blocks, printing appropriate error messages or results.

Input and Output Format (in order of execution):

Division Operation:

Input:

- First line prompts "Enter the numerator: " followed by an integer input for the numerator.
- The second line prompts "Enter the denominator: " followed by an integer input for the denominator.

Output:

- If the denominator is 0, print: "ArithmeticException: Division by zero is not allowed."
- Otherwise, print: "Result of division: <result>"

Note: Handle `ArithmeticException` for division by zero.

Array Access Operation:

Input:

- Third line prompts "Enter the index to access in the array: " followed by an integer input.
- Use a predefined array {1, 2, 3, 4, 5}.

Output:

- If the index is invalid, print: "ArrayIndexOutOfBoundsException: Index is out of bounds."
- Otherwise, print: "Value at index <index>: <value>"

Note: Handle `ArrayIndexOutOfBoundsException` for invalid indices.

String Length Operation:

Input:

- Fourth line prompts "Enter a string (or 'null' to simulate null pointer exception): " followed by a string input
- If the input is the literal string "null", treat it as a null reference.

Output:

- If the string is null: "NullPointerException: Cannot perform operations on a null reference."
- Otherwise, print: "Length of the string: <length>"

Note:

- Handle `NullPointerException` when attempting to call `.length()` on a null string.
- Refer to the visible test-cases for better understanding.

Source Code:

ExceptionDemo.java

```
import java.util.Scanner;
public class ExceptionDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Scenario 1: Divide by Zero (ArithmeticException)
        System.out.print("Enter the numerator: ");
        int numerator = scanner.nextInt();
        System.out.print("Enter the denominator: ");
        int denominator = scanner.nextInt();

        try {
                        int res = numerator/denominator;
                        System.out.println("Result of division: "+res);
        } catch (ArithmeticException e) {
                        System.out.println("ArithmeticException:
Division by zero is not allowed.");


                // write your code here

        }

        // Scenario 2: Array Index Out of Bounds
(ArrayIndexOutOfBoundsException)
        int[] numbers = { 1, 2, 3, 4, 5 };
        System.out.print("Enter the index to access in the array: ");
        int index = scanner.nextInt();

        try {
                System.out.println("Value at index " +index+ ":
"+numbers[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("ArrayIndexOutOfBoundsException:
Index is out of bounds.");
                // write your code here

        }

        // Scenario 3: Null Pointer Exception (NullPointerException)
        System.out.print("Enter a string (or 'null' to simulate null
pointer exception): ");
        String input = scanner.next();

       String str;
```

```
                if(input.equals("null")){
                        str = null;
                }
                else{
                        str = input;
                }
        // write your code here

         try {

                System.out.println("Length of the string: "+
str.length());

         } catch (NullPointerException n) {
                System.out.println("NullPointerException: Cannot
perform operations on a null reference.");
         }

         scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the numerator:
6
Enter the denominator:
3
Result of division: 2
Enter the index to access in the array:
1
Value at index 1: 2
Enter a string (or 'null' to simulate null pointer exception):
null
NullPointerException: Cannot perform operations on a null reference.

Test Case - 2

User Output
Enter the numerator:
3
Enter the denominator:
0
ArithmeticException: Division by zero is not allowed.
Enter the index to access in the array:
6
ArrayIndexOutOfBoundsException: Index is out of bounds.
Enter a string (or 'null' to simulate null pointer exception):
hello
Length of the string: 5

S.No: 17	Exp. Name: ***Multiple Catch blocks using command line arguments***	**Date: 2026-10-05**

Aim:

Write a Java Program to illustrate **Multiple Catch** blocks using command line argument. Refer to the sample test case provided to write the code.

Input Format:

The program takes two command-line arguments:

- The first argument is the numerator (an integer).
- The second argument is the denominator (an integer).

Output Format:

- If both inputs are valid integers, print the result of the division in the format: "Division: result"
- If the denominator is zero, print: "Cannot divide with zero"
- If the inputs are not valid integers, print: "Input is not of integer type"
- If there are insufficient inputs, print: "Insufficient inputs"

Source Code:

q27498/PrintDivMain.java

```
package q27498;
public class PrintDivMain {
	public static void main(String[] args) {
		try {
		//two command-line arguments converted into integer
values
		//write your code here
		int num=Integer.parseInt(args[0]);
		int den=Integer.parseInt(args[1]);

			try{
			//perform divison operation and catch exception
				int res=num/den;
				System.out.println("Division: "+res);
			//write your code here

			}
			catch(ArithmeticException e){
				System.out.println("Cannot divide with
zero");
		}
	}
	catch(NumberFormatException e){
	System.out.println("Input is not of integer type");
	}
	catch(ArrayIndexOutOfBoundsException e){
		System.out.println("Insufficient inputs");
		//write your code here


}
}
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Input is not of integer type

Test Case - 2
User Output
Division: 0

Test Case - 3
User Output
Cannot divide with zero

Test Case - 4
User Output
Insufficient inputs

S.No: 18	Exp. Name: ***Cube root calculator***	**Date: 2026-10-05**

Aim:

Divijoy is creating a calculator application that returns the cube root of the given number. He wants to write Java code for his application that handles multiple exceptions such as input other than a double or a negative number in a single catch block.

What will be the code that perfectly handles the case that Divijoy wants to perform?

Input Format:

- The input line reads a double value.

Output Format:

- The output is a double representing the cube root of the given number if the user enters the valid input, Otherwise, it will print **An error occurred: Invalid input format** if the user enters non-numeric value or it will print **An error occurred: Cannot calculate cube root of a negative number** if the user enters a negative number.

Source Code:

q28900/CubeRootCalculator.java

```
package q28900;
import java.util.*;
public class CubeRootCalculator {
    // write your code here
        public static void main(String args[]){
    Scanner s=new Scanner(System.in);
                try {
        double num=Double.parseDouble(s.nextLine());
        if(num<0){
                throw new IllegalArgumentException(" Cannot calculate
cube root of a negative number");
        }
        double res=Math.cbrt(num);
        System.out.println(res);
        } catch (InputMismatchException | IllegalArgumentException e) {
        if(e instanceof NumberFormatException){
                System.out.println("An error occurred: Invalid input
format");
        }
        else{
                System.out.println("An error occurred:" +
e.getMessage());}
                }
        }


}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
64 4.0

Test Case - 2
User Output
-64

```
An error occurred: Cannot calculate cube root of a negative number
```

Test Case - 3
User Output
CodeTantra
An error occurred: Invalid input format

S.No: 19	Exp. Name: ***Weight Limit Validation using Custom Exception***	**Date: 2026-10-29**

Aim:

Write a program to handle a user-defined exception for weight validation. Create a custom exception class **InvalidWeight** and use it to check if a given weight input exceeds a specified limit (100 kg). If the weight is valid, display a success message; otherwise, throw and catch the custom exception.

Program Requirements:

1. InvalidWeight Class (Custom Exception):

- Extend the **Exception** class
- Create a parameterized constructor that accepts a String message
- Pass the message to the parent **Exception** class using **super()**

2. **CheckWeight** Class:

- Create a method **validWeight(int weight)** that throws **InvalidWeight**
- If weight > 100, throw new **InvalidWeight** with message: "<weight> is invalid weight\n"
- If weight ≤ 100, print: "<weight> is the valid weight\n"

Input Format:

- First line: Prompt "Enter weight: " followed by an integer input

Output Format:

- If the weight is less than or equal to 100, print: "<weight> is the valid weight"
- If the weight is greater than 100, print: "Exception caught: <weight> is invalid weight".

Note:

- Refer to the visible test cases to strictly match with the input/output layout.
- The main method has been provided to you in the editor, understand and fill in the code accordingly.

Source Code:

CheckWeight.java

```
import java.util.*;
import java.lang.*;
class InvalidWeight extends Exception {
        public InvalidWeight(String message){
                super(message);
        }
}
class CheckWeight {
        public void validWeight(int weight)throws InvalidWeight{
                if(weight>100){
                        throw new InvalidWeight(weight +" is invalid
weight\n");
                }
                else{
                        System.out.println(weight +" is the valid
weight");
                }
                }
        public static void main(String args[]) {
                CheckWeight cw = new CheckWeight();
                Scanner sc = new Scanner(System.in);
                int weight;
                System.out.print("Enter weight: ");
                weight = sc.nextInt();
                try {
                        cw.validWeight(weight);
                } catch(InvalidWeight iw) {
                        System.out.print("Exception caught: " +
iw.getMessage());
                }
        }

}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter weight:

101
Exception caught: 101 is invalid weight

Test Case - 2
User Output
Enter weight:
1000
Exception caught: 1000 is invalid weight

Test Case - 3
User Output
Enter weight:
99
99 is the valid weight

S.No: 20	Exp. Name: ***Producer Consumer Concept***	**Date: 2026-10-31**

Aim:

Write a Java program using synchronized threads, which demonstrates producer consumerconcept.

The Producer-Consumer problem is a classic synchronization problem in computer science. It involves two types of threads:

- **Producer**: This thread generates data (e.g., items) and places it in a shared resource (like a buffer).
- **Consumer**: This thread takes data (items) from the shared resource for processing or consumption.

The problem revolves around coordinating these threads such that:

- The producer does not produce when the buffer is full.
- The consumer does not consume when the buffer is empty.
- Both threads operate safely without corrupting the shared resource.

To achieve synchronization in Java, we can use the synchronized keyword along with wait and notify methods to control the interaction between the threads.

Input Format:

- Prompt the user to enter an integer representing the number of items to produce along with a print statement **"Enter the number of items to produce: "**

Output Format:

Output should print the following:

- Messages indicating the production of each item as <Produced: {Produced item}>
- Messages indicating the consumption of each item as <Consumed: {Consumed item}>

Source Code:

q28802/ProducerConsumerDemo.java

```
package q28802;
import java.util.LinkedList;
import java.util.Scanner;
class  Buffer{
        private LinkedList<Integer>list=new LinkedList<>();
        private final int capacity=1;
        synchronized void produce(int item){
                try{
                        while(list.size()==capacity){
                                wait();}
                        list.add(item);
                        System.out.println("Produced: "+item);
                        notifyAll();
                }catch(InterruptedException e){
                Thread.currentThread().interrupt();
                }}
        synchronized void consume(){
                try{
                        while(list.isEmpty()){
                                wait();}
                        int item=list.removeFirst();
                        System.out.println("Consumed: "+ item);
                        notifyAll();
                }catch(InterruptedException e){
                        Thread.currentThread().interrupt();
                }
        }
}
class Producer extends Thread{
        private Buffer buffer;
        private int n;
        Producer(Buffer buffer,int n){
                this.buffer=buffer;
                this.n=n;
        }
        public void run(){
                for(int i=1;i<=n;i++){
                        buffer.produce(i);
                }
        }
}
class Consumer extends Thread{
        private Buffer buffer;
        private int n;
```

```
        Consumer(Buffer buffer,int n){
                this.buffer=buffer;
                this.n=n;
        }
        public void run(){
                for(int i=1;i<=n;i++){
                        buffer.consume();
                }
        }}
public class ProducerConsumerDemo {
        public static void main(String args[])throws
InterruptedException{
                Scanner sc=new Scanner(System.in);
                System.out.println("Enter the number of items to
produce: ");
                int n=sc.nextInt();
                Buffer buffer=new Buffer();
                Producer producer=new Producer(buffer,n);
                Consumer consumer=new Consumer(buffer,n);
                producer.start();
                consumer.start();
                producer.join();
                consumer.join();
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of items to produce:
3
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3

Test Case - 2
User Output
Enter the number of items to produce:
4
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4

S.No: 21	Exp. Name: ***Multithreaded Java Program***	**Date: 2026-10-31**

Aim:

Implement a multithreaded Java program where three threads, with a specified number of times as input by the user, execute concurrently with a fixed sleep time before each message print.

Requirements:

1. Define the GoodMorning thread class

1. Create a class named **GoodMorning** that extends `Thread`.
2. Accept the number of repetitions as a parameter through the constructor.
3. Override the `run()` method to:
    - Pause execution for 0.2 seconds using `Thread.sleep()`.
    - Print "Good morning" for the given number of repetitions.

2. Define the Hello thread class

1. Create a class named **Hello** that extends `Thread`.
2. Accept the number of repetitions as a parameter through the constructor.
3. Override the `run()` method to:
    - Pause execution for 0.3 seconds using `Thread.sleep()`.
    - Print "Hello" after each sleep.
    - Repeat for the given number of times.

3. Define the Welcome thread class

1. Create a class named **Welcome** that extends `Thread`.
2. Accept the number of repetitions as a parameter through the constructor.
3. Override the `run()` method to:
    - Pause execution for 0.4 seconds using `Thread.sleep()`.
    - Print "Welcome" after each sleep.
    - Repeat for the given number of times.

4. Implement the **MultithreadDemo** class

1. Use a `Scanner` object to accept three integers from the user:
    - Number of times "Good morning" should be printed.
    - Number of times "Hello" should be printed.
    - Number of times "Welcome" should be printed.
2. Create objects of **GoodMorning**, **Hello**, and **Welcome** classes using the user inputs.
3. Start all three threads using the `start()` method.

Input Format:

Three integers entered by the user (each on a separate line):

- Repetitions for "Good morning".
- Repetitions for "Hello".
- Repetitions for "Welcome".

Output Format:

- The program prints "Good morning", "Hello", and "Welcome" concurrently.
- Each message is printed after its respective sleep time for the given number of repetitions.
- Since threads run concurrently, the order of output may vary.

Source Code:

q77401/MultithreadDemo.java

```
package q77401;
import java.util.Scanner;

class GoodMorning extends Thread {
        // write your code here
        int n;
    GoodMorning(int n){
                this.n=n;
        }
    public synchronized void run() {
        try {
                        for(int i=0;i<n;i++){
                                Thread.sleep(200);
                                System.out.println("Good morning");
                        }}
        catch (Exception e) {
            System.out.println(e);
        }
    }
}

class Hello extends Thread {
    // write your code here
    int n;
        Hello(int n){
                this.n=n;
        }
    public synchronized void run() {
        try {
                        for(int i=0;i<n;i++){
                                Thread.sleep(300);
                                System.out.println("Hello");
                        }
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}

class Welcome extends Thread {
        int n;
        Welcome(int n){
                this.n=n;
```

```
        }

    public synchronized void run() {
        try {
                        for(int i=0;i<n;i++){
Thread.sleep(400);
                                System.out.println("Welcome");
                        }
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}

public class MultithreadDemo {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);

        // Take inputs
        int gmCount = scanner.nextInt(); // repetitions for Good
Morning
        int helloCount = scanner.nextInt(); // repetitions for Hello
        int welcomeCount = scanner.nextInt(); // repetitions for
Welcome
        // write your code here
        GoodMorning gm=new GoodMorning(gmCount);
                Hello h=new Hello(helloCount);
                Welcome w=new Welcome(welcomeCount);
        gm.start();
                h.start();
                w.start();


        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

4
Good morning
Hello
Welcome
Good morning
Hello
Welcome
Hello
Welcome
Welcome

Test Case - 2
User Output
5
3
2
Good morning
Hello
Welcome
Good morning
Hello
Good morning
Welcome
Good morning
Hello
Good morning

S.No: 22	Exp. Name: ***A Java program to illustrate creation of threads using Runnable***	**Date: 2026-10-31**

Aim:

Write a **java** program to illustrate creation of threads using **Runnable** Interface(start method start each of the newly created thread. Inside the `run` method there is `sleep()` for suspend the thread for **500** milliseconds).

Sample Input and Sample Output:

```
Good Morning
Good Morning
Hello
Good Morning
Hello
Welcome
Good Morning
Hello
Welcome
Good Morning
Hello
Welcome
Hello
Welcome
Welcome
```

Source Code:

q19/MyThread.java

```
package q19;
public class MyThread implements Runnable{
        public void run(){
                try{
                        Thread.sleep(500);
                System.out.println("Good Morning");
                System.out.println("Hello");
                System.out.println("Welcome");
        }catch(InterruptedException e){
                        System.out.println(e);
        }
}
        public static void main(String args[]){
                MyThread obj=new MyThread();
                Thread t1=new Thread(obj);
                Thread t2=new Thread(obj);
                Thread t3=new Thread(obj);
                Thread t4=new Thread(obj);
                Thread t5=new Thread(obj);
                t1.start();
                t2.start();
                t3.start();
                t4.start();
                t5.start();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Good Morning
Good Morning
Hello
Good Morning
Hello
Welcome
Good Morning
Hello

Welcome
Good Morning
Hello
Welcome
Hello
Welcome
Welcome

S.No: 23	Exp. Name: ***Daemon Threads***	Date: 2026-10-31

Aim:

Create a Java program that demonstrates the use of user and daemon threads.
Implement a class **WorkerThread** that extends **Thread** and overrides the **run** method to simulate some work by printing messages and sleeping for a short duration.

In the **DaemonThread** class:
1. Prompt the user to input the number of user threads and daemon threads.
2. Create and start the specified number of user threads and daemon threads.
    - **User Threads:** These threads will run to completion and are not interrupted by the main thread.**Daemon Threads:** These threads will run in the background and may be terminated abruptly when the main thread exits.
3. The main thread should print a message indicating its exit after creating the threads.

Input Format:

- The first line contains an integer specifying the number of user threads.
- The second line contains an integer specifying the number of daemon threads.

Output Format:

The output should display the following:

- Messages indicating the **start** and **finish** of each user thread.
- Messages indicating the **start** and **finish** of each daemon thread.
- A message indicating that the main thread is exiting as: "Main thread exiting".

Note:

- Ensure that daemon threads are correctly set using **setDaemon(true)** before calling `start()`.
- Refer to visible test cases for better understanding and to match with input/output layout.

Source Code:

DaemonThread.java

```
import java.util.Scanner;

class WorkerThread extends Thread {
        WorkerThread(String name){
                        super(name);
}
public void run(){
        try{
        System.out.println(getName() + " started");
                if(isDaemon()){
                        Thread.sleep(100);
                }else{
        Thread.sleep(3000);}
        System.out.println(getName() + " finished");
        }catch(InterruptedException e){
                        System.out.println(e);
                }
                }
}
public class DaemonThread {
        public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of user threads: ");
        int numUserThreads = scanner.nextInt();

        System.out.print("Enter the number of daemon threads: ");
        int numDaemonThreads = scanner.nextInt();
                for(int i=1;i<=numUserThreads;i++){
                        WorkerThread t=new WorkerThread("Thread " + i);
                        t.setDaemon(false);
                        t.start();
                }
                for(int i=1;i<=numDaemonThreads;i++){
                        WorkerThread t=new WorkerThread("Thread "+i);
                        t.setDaemon(true);
                        t.start();
                }



        System.out.println("Main thread exiting");
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of user threads:
3
Enter the number of daemon threads:
2
Main thread exiting
Thread 2 started
Thread 3 started
Thread 1 started
Thread 2 started
Thread 1 started
Thread 1 finished
Thread 1 finished
Thread 2 finished
Thread 2 finished
Thread 3 finished

Test Case - 2
User Output
Enter the number of user threads:
2
Enter the number of daemon threads:
3
Main thread exiting
Thread 3 started
Thread 2 started
Thread 1 started
Thread 1 started
Thread 2 started
Thread 3 finished
Thread 2 finished
Thread 1 finished
Thread 1 finished
Thread 2 finished

S.No: 24	Exp. Name: ***Thread Execution and Synchronization in Java***	**Date: 2026-11-02**

Aim:

Write a Java program that creates two threads, **FirstThread** and **SecondThread**, to print messages a specified number of times with a 100 ms delay.

- The program should prompt the user to enter the number of messages for the First Thread and the Second Thread.
- **FirstThread** should print messages as "Message from First Thread: <i>" where <i> is the current iteration.
- **SecondThread** should print messages as "Message from Second Thread: <i>" where <i> is the current iteration.
- The main thread should:
    - Start both threads using `start()`.
    - Display their alive status using `isAlive()`.
    - Wait for each thread to finish using `join()`.
    - Display messages while waiting and after completion.
    - Verify and display the alive status of threads after they complete.
    - Print a final message indicating the main thread has finished.
- Both threads should properly handle `InterruptedException` during sleep.

Input Format:

```
Enter the count for First Thread: [user-input-integer]
Enter the count for Second Thread: [user-input-integer]
```

Output Format:

```
FirstThread is alive: true
SecondThread is alive: true
Waiting for FirstThread to finish.
Message from First Thread: 1
Message from Second Thread: 1
Message from First Thread: 2
Message from Second Thread: 2
...
FirstThread has finished.
Waiting for SecondThread to finish.
SecondThread has finished.
FirstThread is alive after join: false
SecondThread is alive after join: false
Main thread has finished.
```

Note:

- The print statements for the output are already present in the editor. Your task is to implement the logic in the specified sections of the code.
- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q20/ThreadDemo.java

```
package q20;
import java.util.Scanner;
class FirstThread extends Thread {
        int count;
        FirstThread(int count){
                this.count=count;
        }
        public void run(){
                for(int i=1;i<=count;i++){
                        try{
                                Thread.sleep(100);
                                System.out.println("Message from First
Thread: "+i);
                        }catch(InterruptedException e){
                                Thread.currentThread().interrupt();
                                return;
                        }
                }
        }
}
class SecondThread extends Thread {
        int count;
        SecondThread(int count){
                this.count=count;
        }
        public void run(){
                for(int i=1;i<=count;i++){
                        try{
                                Thread.sleep(100);
                                System.out.println("Message from Second
Thread: "+i);
                        }catch(InterruptedException e){
                                Thread.currentThread().interrupt();
                                return;
                        }
                }
        }
}
public class ThreadDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the count for First Thread: ");
        int count1 = scanner.nextInt();
```

```
        System.out.print("Enter the count for Second Thread: ");
        int count2 = scanner.nextInt();

        FirstThread firstThread = new FirstThread(count1);
        SecondThread secondThread = new SecondThread(count2);

        firstThread.start();
        secondThread.start();
        System.out.println("FirstThread is alive: "
+firstThread.isAlive());
        System.out.println("SecondThread is alive: "
+secondThread.isAlive());

        try {
            // Wait for threads to finish
            System.out.println("Waiting for FirstThread to finish.");
                        firstThread.join();
            System.out.println("FirstThread has finished.");
                        System.out.println("Waiting for SecondThread to
finish.");
                        secondThread.join();
            System.out.println("SecondThread has finished.");
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted: " + e);
        }

        // Check again if threads are alive after join
        System.out.println("FirstThread is alive after join: "
+firstThread.isAlive());
        System.out.println("SecondThread is alive after join: "
+secondThread.isAlive());

        System.out.println("Main thread has finished.");
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the count for First Thread:

5
Enter the count for Second Thread:
5
Message from Second Thread: 1
Message from First Thread: 1
FirstThread is alive: true
SecondThread is alive: true
Waiting for FirstThread to finish.
Message from Second Thread: 2
Message from First Thread: 2
Message from Second Thread: 3
Message from First Thread: 3
Message from Second Thread: 4
Message from First Thread: 4
Message from Second Thread: 5
Message from First Thread: 5
FirstThread has finished.
Waiting for SecondThread to finish.
SecondThread has finished.
FirstThread is alive after join: false
SecondThread is alive after join: false
Main thread has finished.

Test Case - 2
User Output
Enter the count for First Thread:
3
Enter the count for Second Thread:
7
FirstThread is alive: true
Message from First Thread: 1
Message from Second Thread: 1
SecondThread is alive: true
Waiting for FirstThread to finish.
Message from First Thread: 2
Message from Second Thread: 2
Message from First Thread: 3
Message from Second Thread: 3
FirstThread has finished.
Waiting for SecondThread to finish.

Message from Second Thread: 5
Message from Second Thread: 6
Message from Second Thread: 7
SecondThread has finished.
FirstThread is alive after join: false
SecondThread is alive after join: false
Main thread has finished.

Test Case - 3
User Output
Enter the count for First Thread:
2
Enter the count for Second Thread:
10
Message from Second Thread: 1
Message from First Thread: 1
FirstThread is alive: true
SecondThread is alive: true
Waiting for FirstThread to finish.
Message from Second Thread: 2
Message from First Thread: 2
Message from Second Thread: 3
FirstThread has finished.
Waiting for SecondThread to finish.
Message from Second Thread: 4
Message from Second Thread: 5
Message from Second Thread: 6
Message from Second Thread: 7
Message from Second Thread: 8
Message from Second Thread: 9
Message from Second Thread: 10
SecondThread has finished.
FirstThread is alive after join: false
SecondThread is alive after join: false
Main thread has finished.

S.No: 25	Exp. Name: ***Arithmetic operations using packages***	**Date: 2026-11-02**

Aim:

Write a java program to create a user-defined package called "**data**" and perform **Arithmetic operations** (addition, subtraction, multiplication, divison) using packages.

Note: Write the required operations in the data/Arithmetic.java file.

Source Code:

data/Arithmetic.java

```
package data;

public class Arithmetic {

    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b) {
        return a - b;
    }

    public static int multiply(int a, int b) {
        return a * b;
    }

    public static double divide(int a, int b) {
        return (double) a / b;
    }
}
```

ArithmeticMain.java

```
import data.Arithmetic;
import java.util.*;
public class ArithmeticMain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("First number: ");
        int a = sc.nextInt();
        System.out.print("Second number: ");
        int b = sc.nextInt();
        System.out.println("Sum: " + Arithmetic.add(a, b));
        System.out.println("Difference: " + Arithmetic.subtract(a, b));
        System.out.println("Product: " + Arithmetic.multiply(a, b));
        if (b != 0) {
            System.out.println("Divison: " + Arithmetic.divide(a, b));
        } else {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
First number:
12
Second number:
10
Sum: 22
Difference: 2
Product: 120
Divison: 1.2

Test Case - 2
User Output
First number:
25

Second number:
0
Sum: 25
Difference: 25
Product: 0
Cannot divide by zero